

# Transformer model

Tran Hong Viet  
UET, VNU Hanoi

# Rise of Transformers

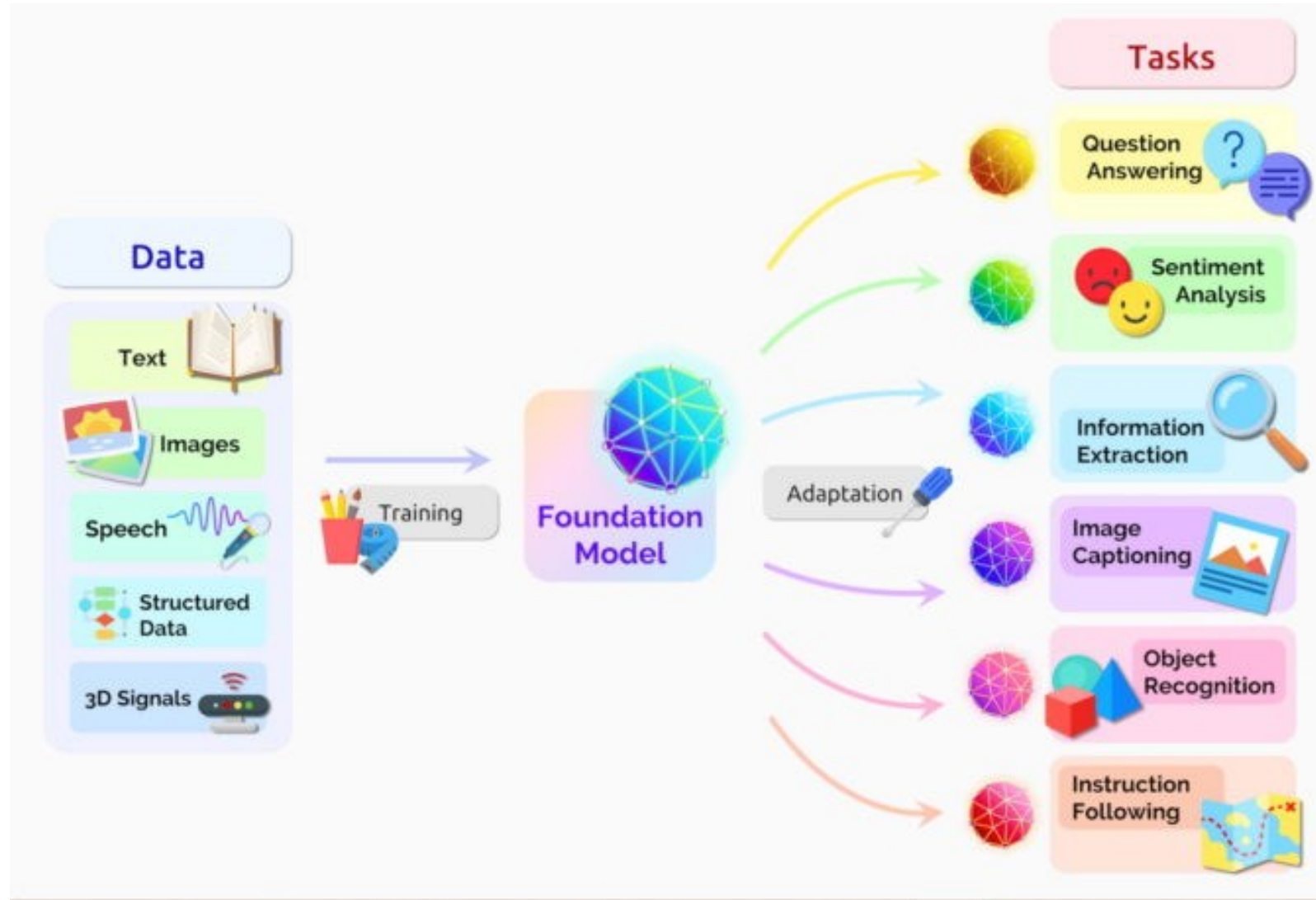
## Natural Language Processing

| Model                       | Description                                                                                                          | Reference                        |
|-----------------------------|----------------------------------------------------------------------------------------------------------------------|----------------------------------|
| <a href="#">ALBERT</a>      | A Lite BERT for Self-supervised Learning of Language Representations                                                 | <a href="#">arXiv:1909.11942</a> |
| <a href="#">BERT</a>        | A powerful pre-trained language representation model: BERT (Bidirectional Encoder Representations from Transformers) | <a href="#">arXiv:1810.04805</a> |
| <a href="#">NHNet</a>       | A transformer-based multi-sequence to sequence model: Generating Representative Headlines for News Stories           | <a href="#">arXiv:2001.09386</a> |
| <a href="#">Transformer</a> | A transformer model to translate the WMT English to German dataset                                                   | <a href="#">arXiv:1706.03762</a> |
| <a href="#">XLNet</a>       | XLNet: Generalized Autoregressive Pretraining for Language Understanding                                             | <a href="#">arXiv:1906.08237</a> |

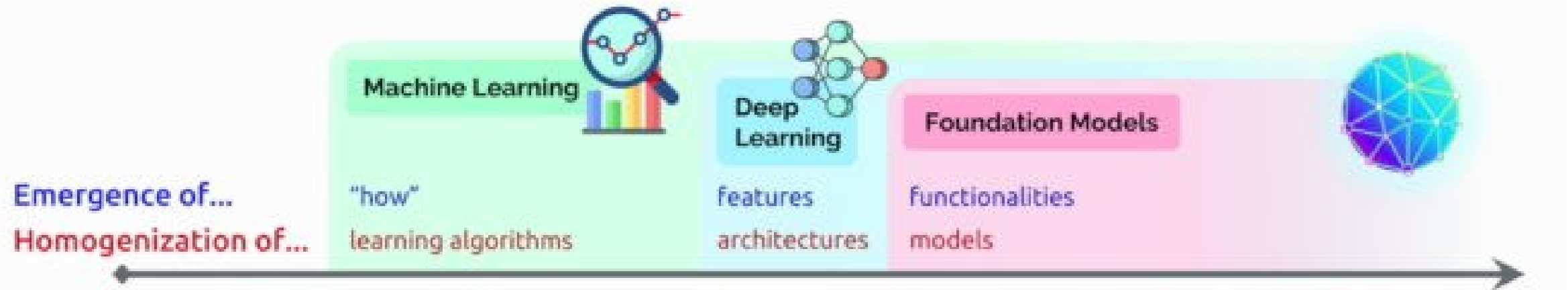
**ChatGPT, GPT3** (OpenAI)

- **Computer Vision**
  - Transformer cartoon (DALL-E)

# What Can Transformer Models Do?



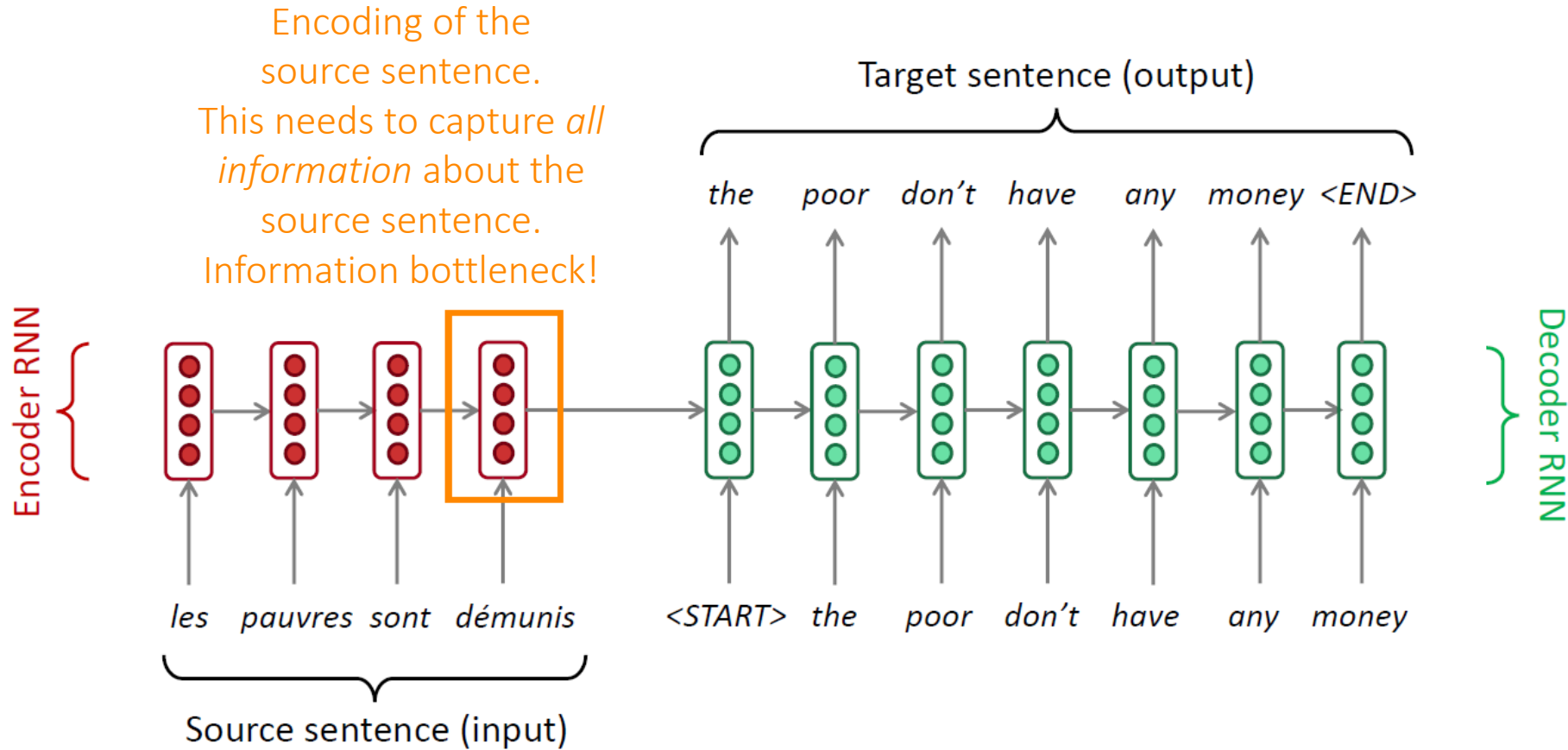
# The Cycle of Transformer AI



# Sequence-to-sequence is everywhere!

- Sequence-to-sequence is useful for *more than just MT*
- Many NLP tasks can be phrased as sequence-to-sequence:
  - Summarization (long text → short text)
  - Dialogue (previous utterances → next utterance)
  - Parsing (input text → output parse as sequence)
  - Code generation (natural language → Python code)

# Sequence-to-sequence: the bottleneck problem

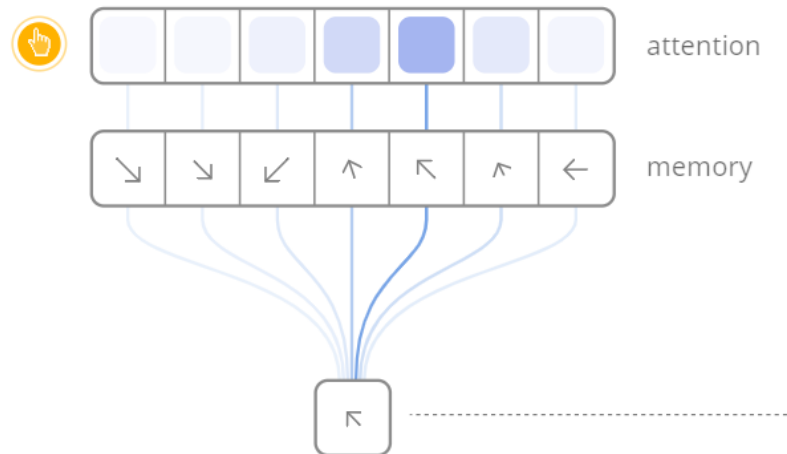


# Attention

- **Attention** provides a solution to the bottleneck problem.
- **Core idea:** on each step of the decoder, *focus on a particular part* of the source sequence

# Attention is a *general* Deep Learning technique

- More general definition of attention
  - Given a set of vector *values*, and a vector *query*, **attention** is a technique to compute a **weighted sum of the values**, dependent on the query.
  - We sometimes say that the query *attends to* the values
  - For example, in the seq2seq + attention model, each decoder hidden state *attends to* the encoder hidden states

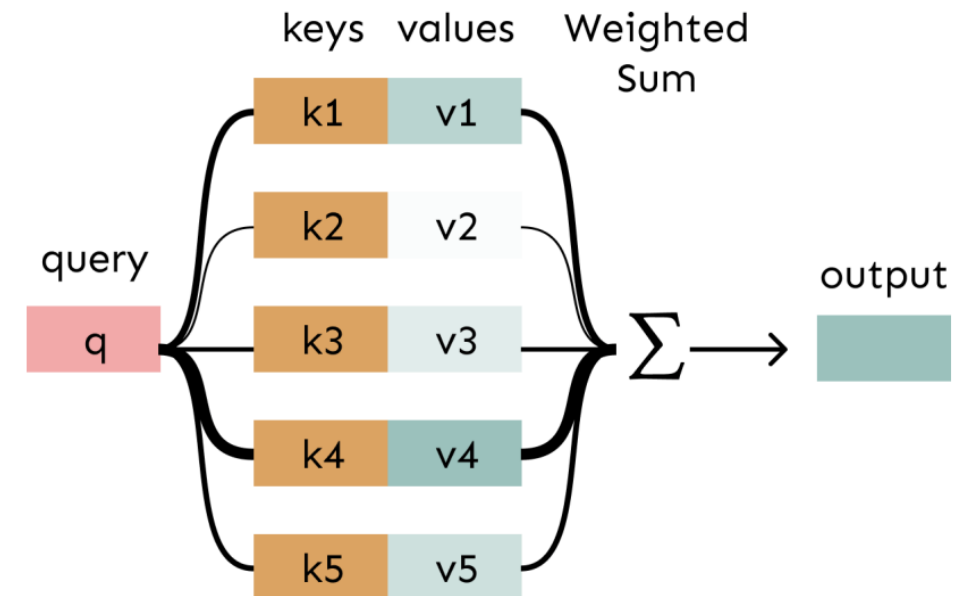
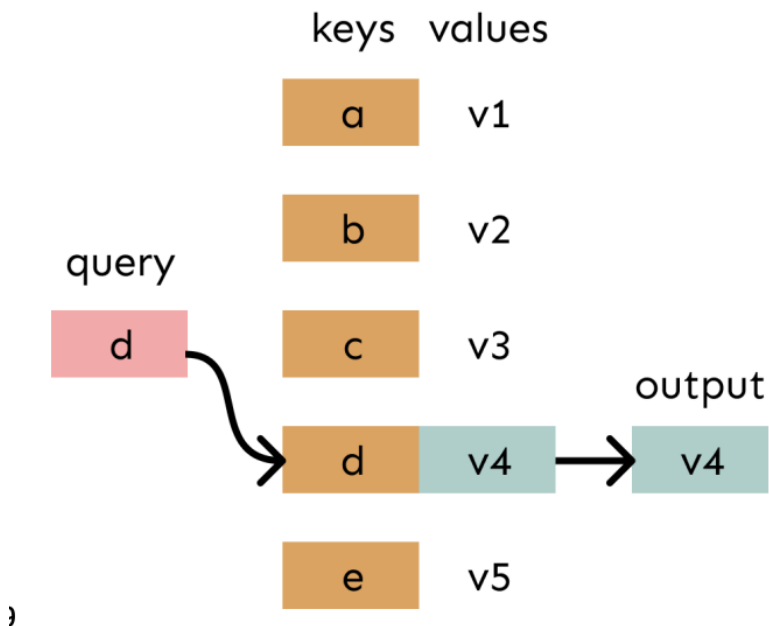


# Attention as a soft, averaging lookup table

- We can think of **attention** as performing fuzzy lookup a in **key-value store**

**Lookup table:** a table of keys that map to values. The query matches one of the keys, returning its value.

**Attention:** The query matches to all keys softly to a weight between 0 and 1. The keys' values are multiplied by the weights and summed.



# Problems with RNNs = Motivation for Transformers

- Recurrent models typically factor computation along the symbol positions of the input and output sequences
  - Sequential computation prevents parallelization
  - Critical at longer sequence lengths, as memory constraints limit batching across examples
- Despite advanced RNNs like LSTMs, RNNs still need attention mechanism to deal with long range dependencies – path length for codependent computation between states grows with sequence
- But if attention gives us access to any state... maybe we don't need the RNN?

# Transformers (2017)

---

## Attention Is All You Need

---

**Ashish Vaswani\***  
Google Brain  
avaswani@google.com

**Noam Shazeer\***  
Google Brain  
noam@google.com

**Niki Parmar\***  
Google Research  
nikip@google.com

**Jakob Uszkoreit\***  
Google Research  
usz@google.com

**Llion Jones\***  
Google Research  
llion@google.com

**Aidan N. Gomez\* †**  
University of Toronto  
aidan@cs.toronto.edu

**Lukasz Kaiser\***  
Google Brain  
lukaszkaizer@google.com

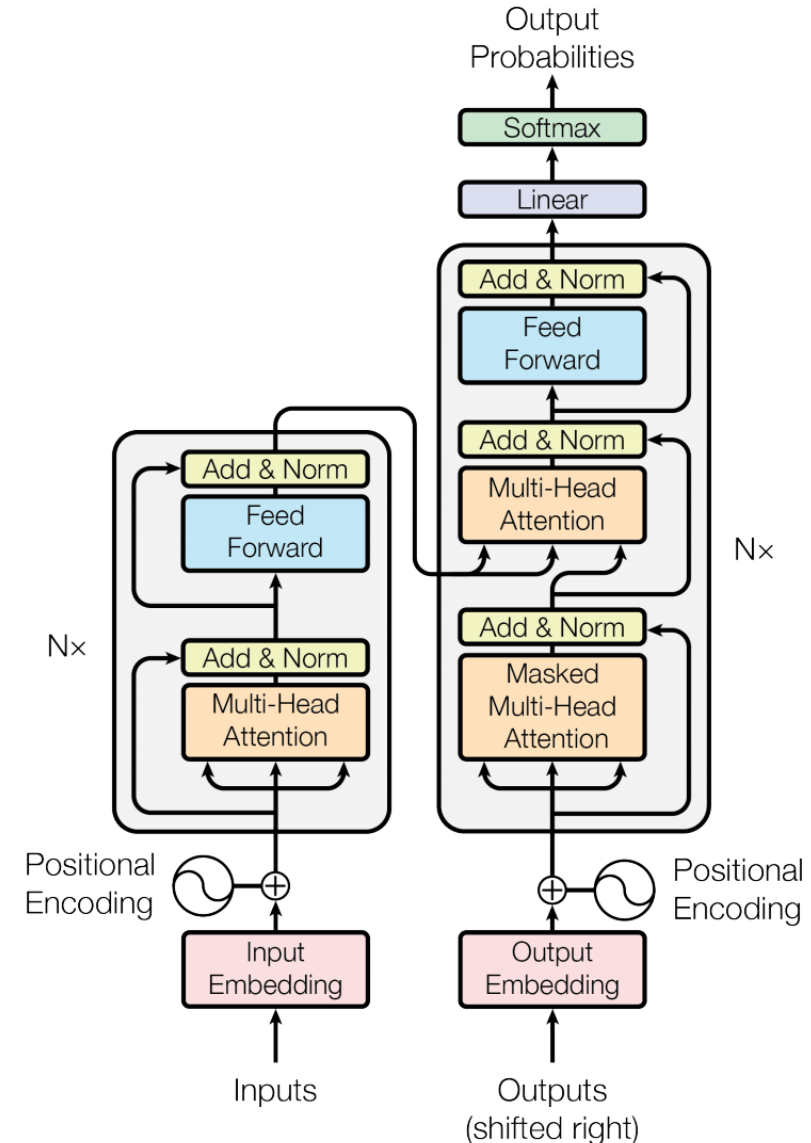
**Illia Polosukhin\* †**  
illia.polosukhin@gmail.com



(Vaswani et al., 2017)

# Transformer Overview

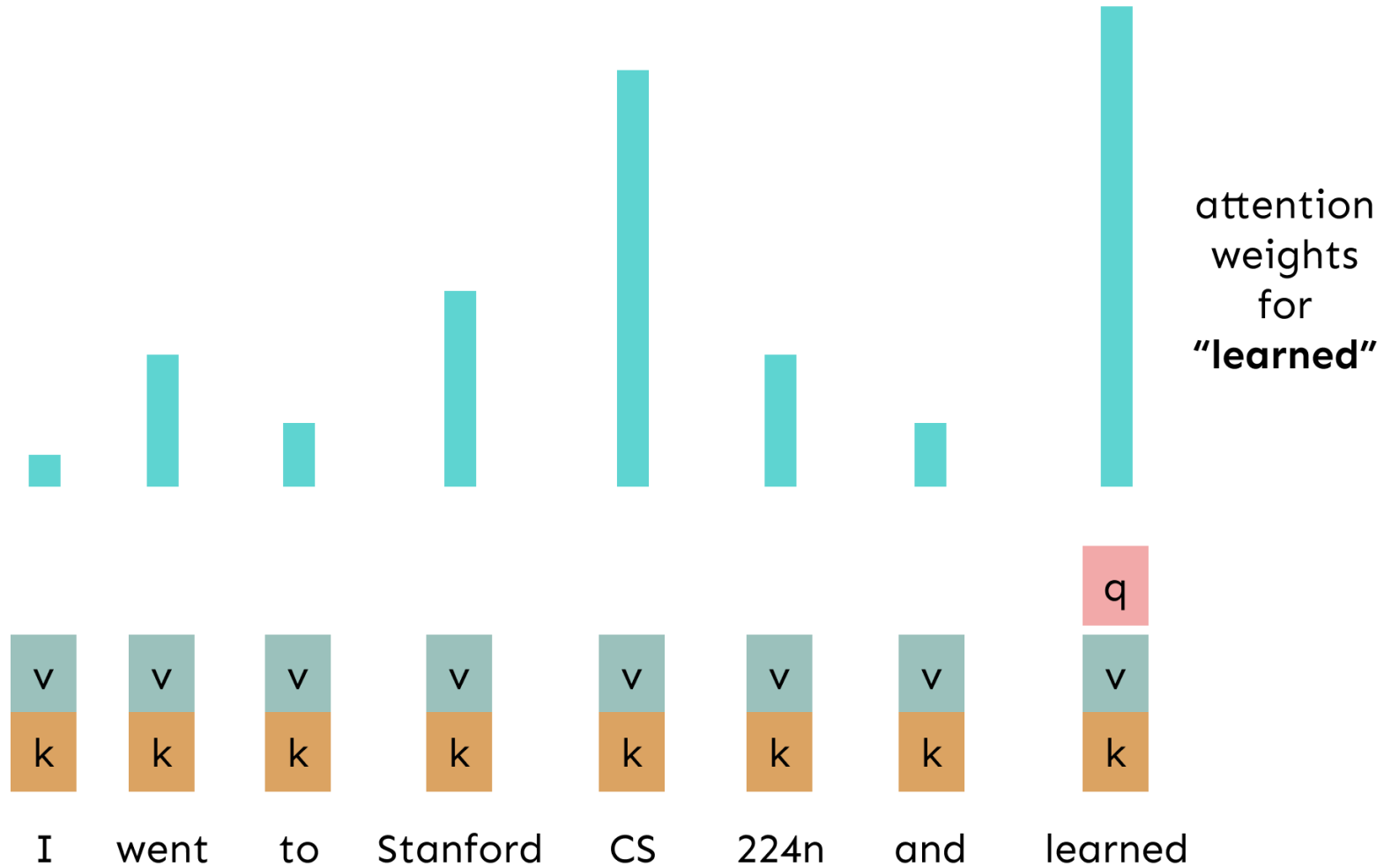
- Sequence-to-sequence
- Encoder-Decoder
- Task: machine translation with parallel corpus
- Predict each translated word
- Final cost/error function is standard cross-entropy error on top of a softmax classifier



# Transformer Basics

- Let's define the basic building blocks of transformer networks first: new attention layers!

# Seft-attention Example



# Self-Attention: keys, queries, values from the same sequence

Let  $\mathbf{w}_{1:n}$  be a sequence of words in vocabulary  $V$ , like *Zuko made his uncle tea*.

For each  $\mathbf{w}_i$ , let  $\mathbf{x}_i = E\mathbf{w}_i$ , where  $E \in \mathbb{R}^{d \times |V|}$  is an embedding matrix.

1. Transform each word embedding with weight matrices  $Q, K, V$ , each in  $\mathbb{R}^{d \times d}$

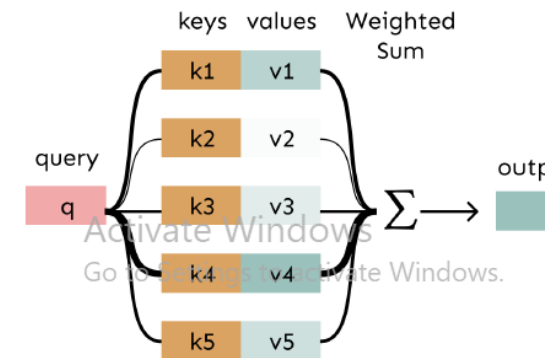
$$\mathbf{q}_i = Q\mathbf{x}_i \text{ (queries)} \quad \mathbf{k}_i = K\mathbf{x}_i \text{ (keys)} \quad \mathbf{v}_i = V\mathbf{x}_i \text{ (values)}$$

2. Compute pairwise similarities between keys and queries; normalize with softmax

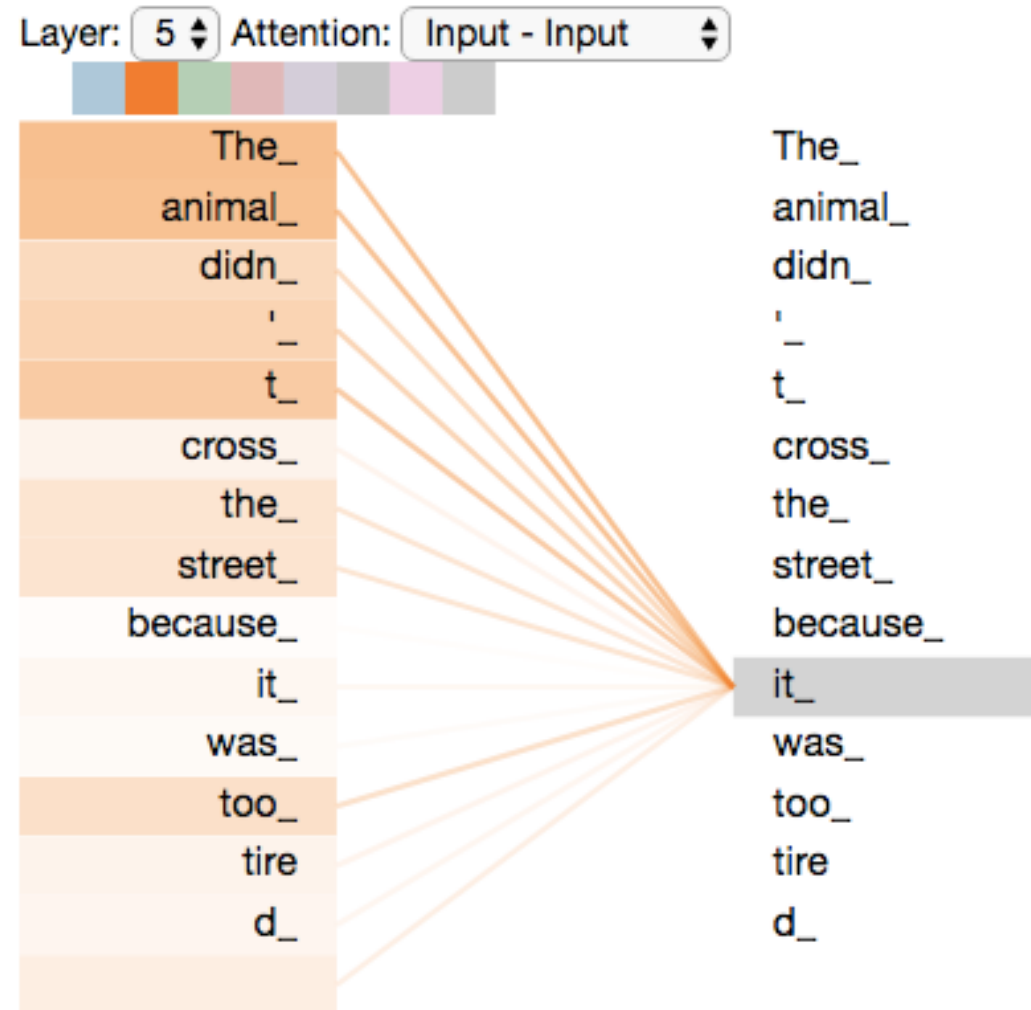
$$\mathbf{e}_{ij} = \mathbf{q}_i^\top \mathbf{k}_j \quad \alpha_{ij} = \frac{\exp(\mathbf{e}_{ij})}{\sum_{j'} \exp(\mathbf{e}_{ij'})}$$

3. Compute output for each word as weighted sum of values

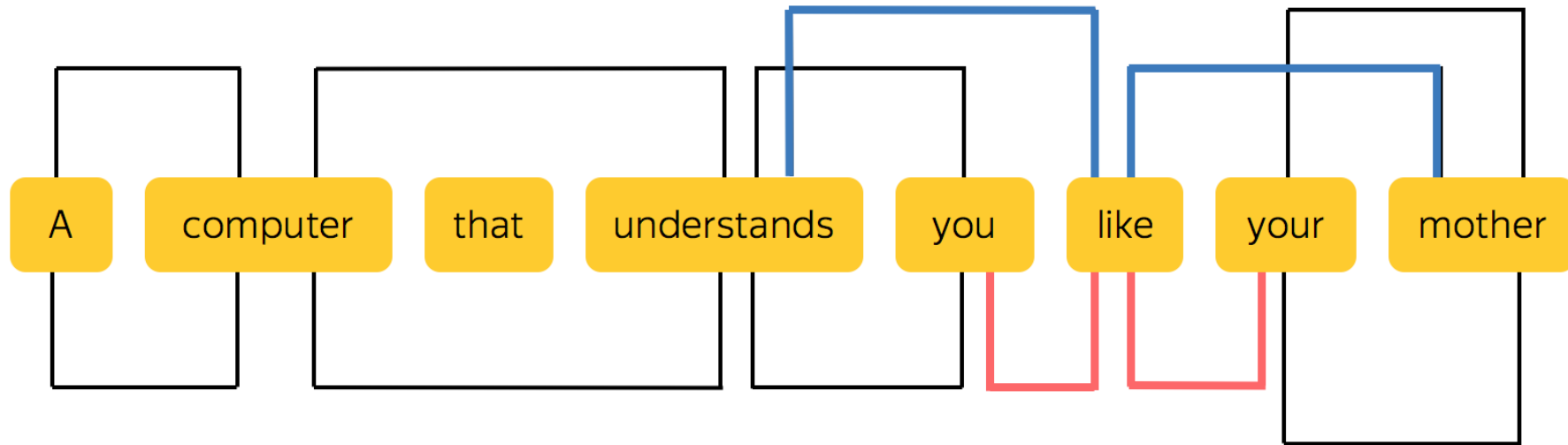
$$\mathbf{o}_i = \sum_j \alpha_{ij} \mathbf{v}_j$$



# Self Attention Visualization



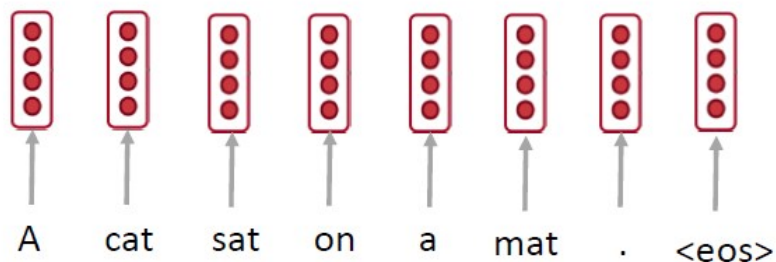
# Self-Attention



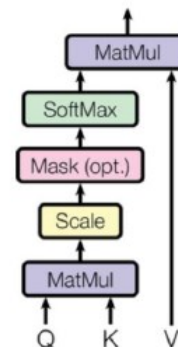
(example and picture from David Talbot)

# Self-attention: A Running Example

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

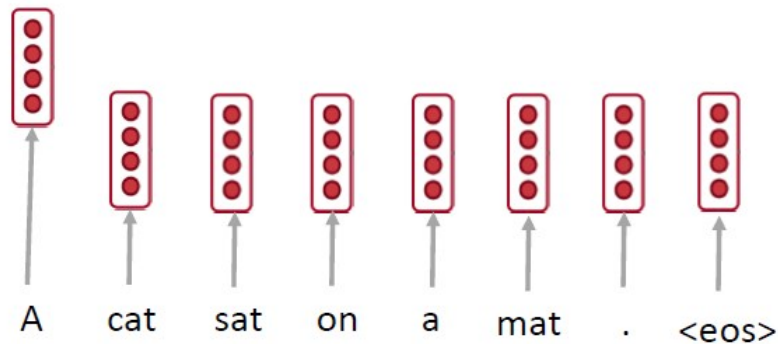


Scaled Dot-Product Attention

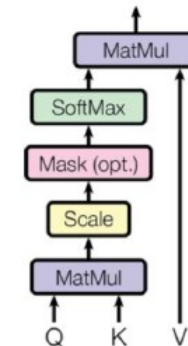


# Self-attention: A Running Example

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

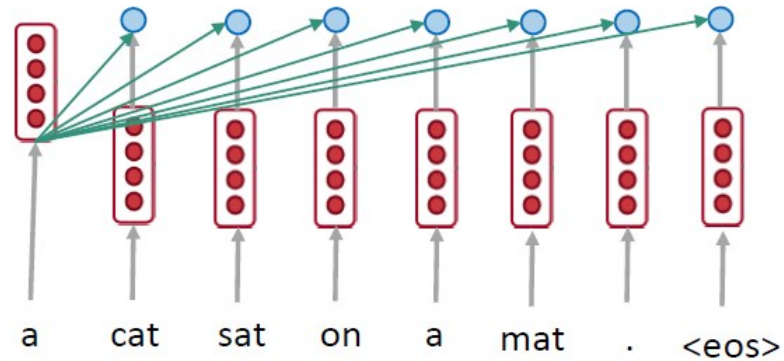


Scaled Dot-Product Attention

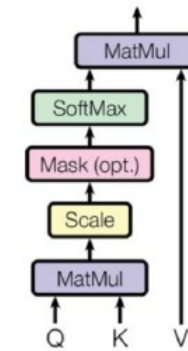


# Self-attention: A Running Example

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

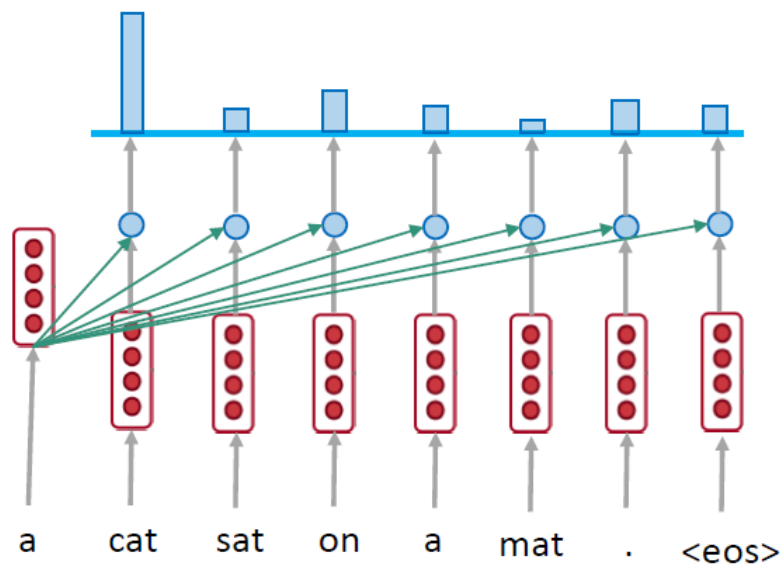


Scaled Dot-Product Attention

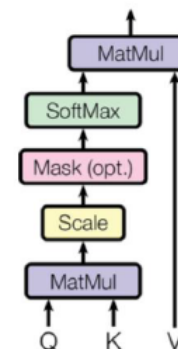


# Self-attention: A Running Example

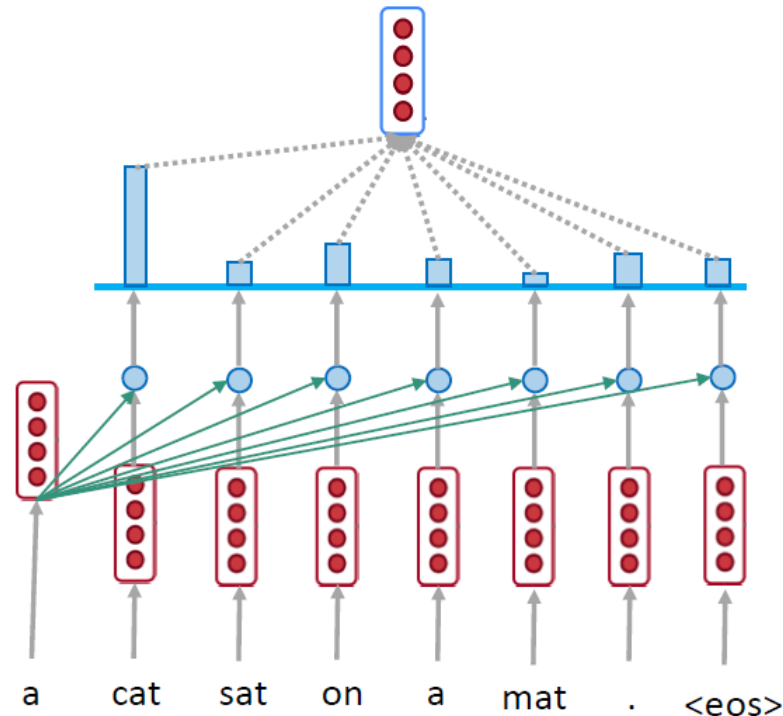
$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$



Scaled Dot-Product Attention

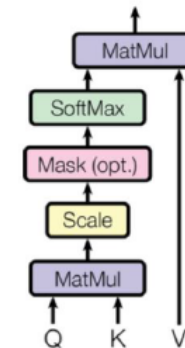


# Self-attention: A Running Example



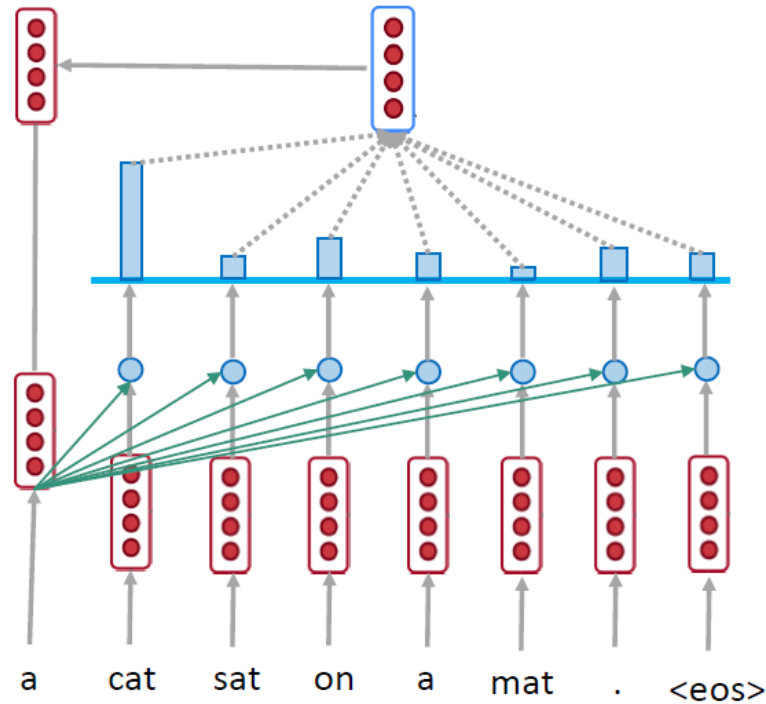
$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

Scaled Dot-Product Attention



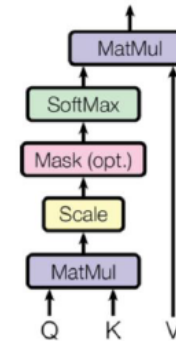
# Self-attention: A Running Example

update  
representation  
for the word "a"

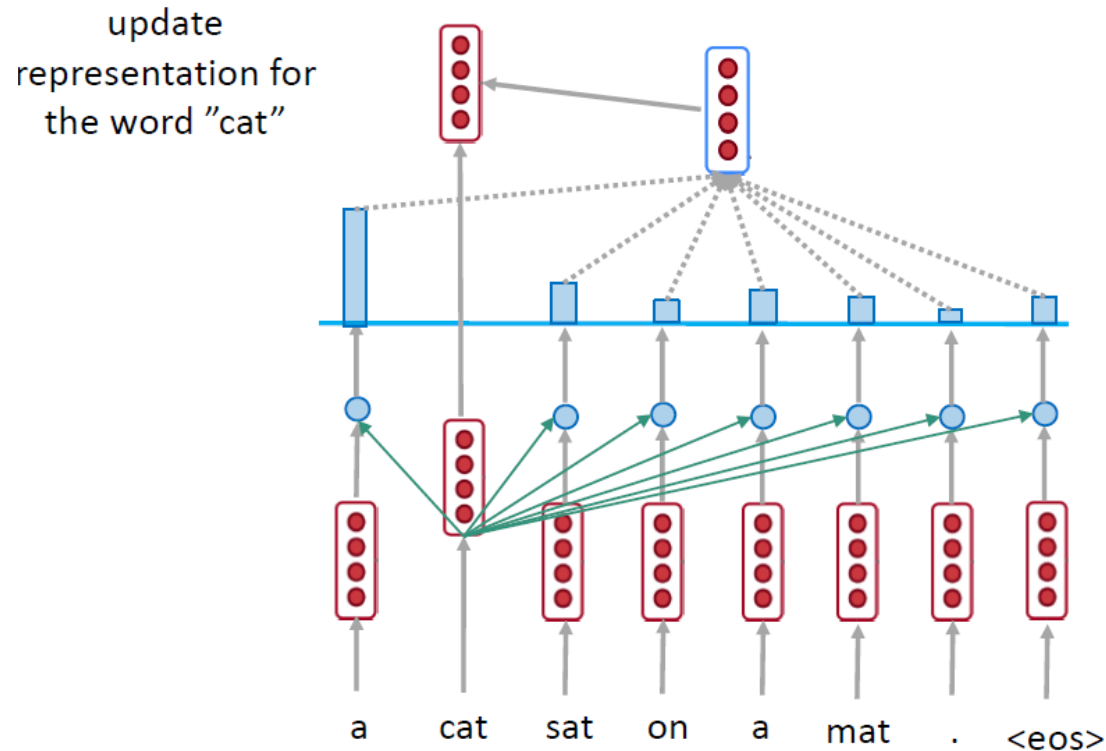


$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

Scaled Dot-Product Attention

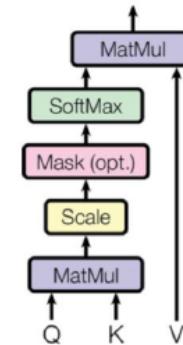


# Self-attention: A Running Example



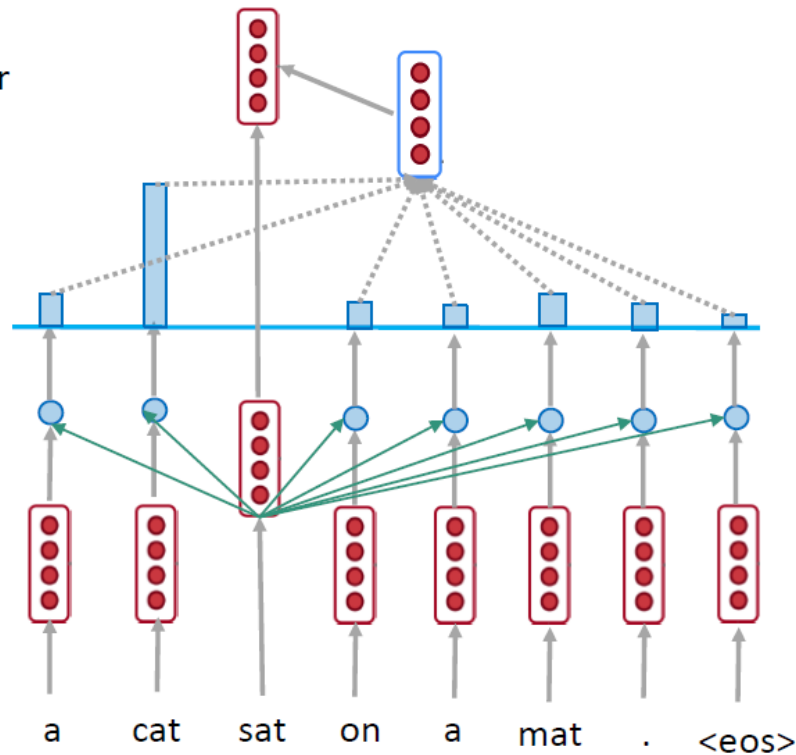
$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

Scaled Dot-Product Attention



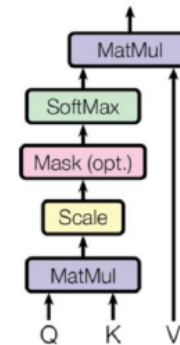
# Self-attention: A Running Example

update  
representation for  
the word "sat"



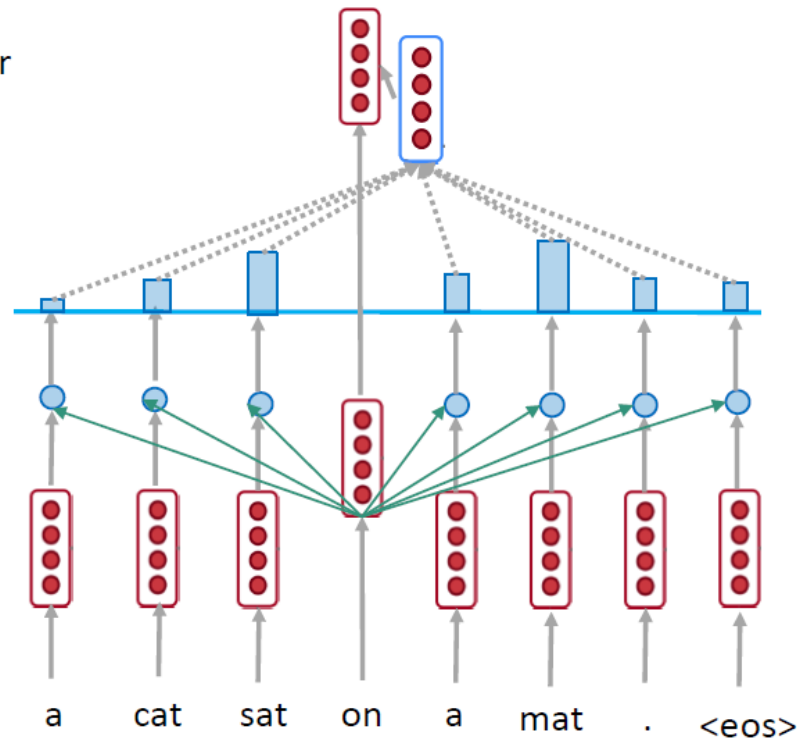
$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

Scaled Dot-Product Attention



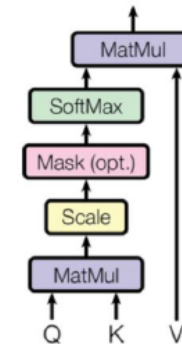
# Self-attention: A Running Example

update  
representation for  
the word "on"



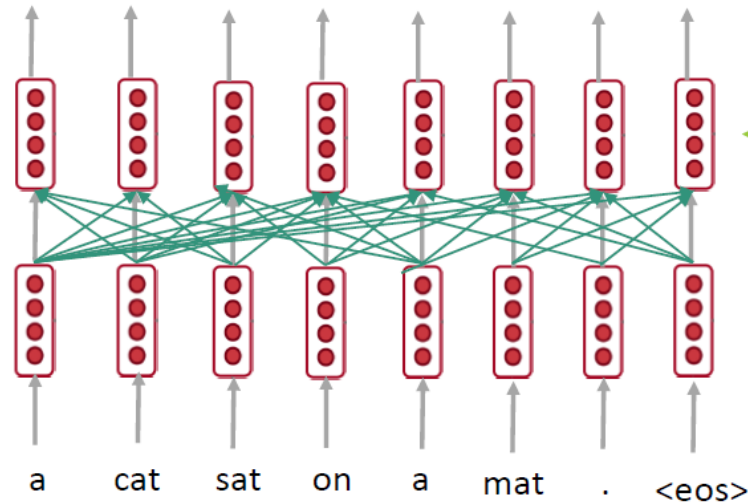
$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

Scaled Dot-Product Attention



# Self-attention: A Running Example

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

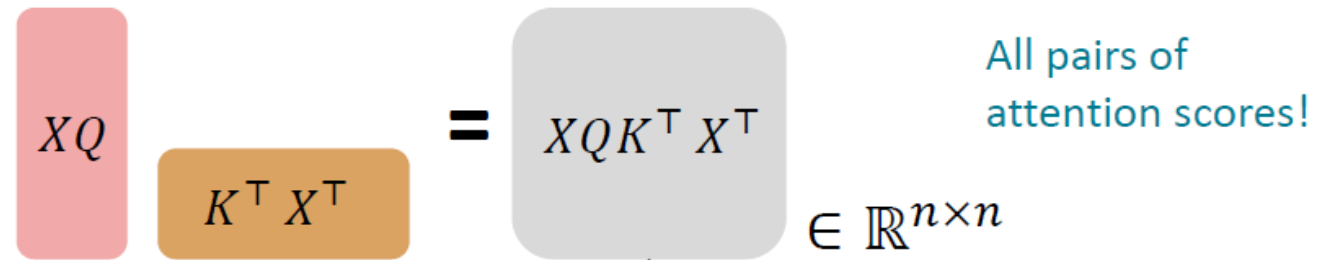


Updated representations  
for each word  
(one layer)

# Sequence-stacked form of Attention

- Let's look at how key-query-value attention is computed, in matrices.
  - Let  $X = [x_1; \dots; x_n] \in \mathbb{R}^{n \times d}$  be the concatenation of input vectors.
  - First, note that  $XK \in \mathbb{R}^{n \times d}$ ,  $XQ \in \mathbb{R}^{n \times d}$ ,  $XV \in \mathbb{R}^{n \times d}$ .
  - The output is defined as  $\text{output} = \text{softmax}(XQ(XK)^\top)XV \in \mathbb{R}^{n \times d}$ .

First, take the query-key dot products in one matrix multiplication:  $XQ(XK)^\top$



A diagram illustrating the first step of attention computation. On the left, a pink vertical rectangle labeled  $XQ$  is multiplied by an orange horizontal rectangle labeled  $K^\top X^\top$ . An equals sign follows, leading to a gray rounded rectangle labeled  $XQK^\top X^\top$ . To the right of this rectangle is the text  $\in \mathbb{R}^{n \times n}$ . Further to the right, the text "All pairs of attention scores!" is written in blue.

Next, softmax, and compute the weighted average with another matrix multiplication.



A diagram illustrating the second step of attention computation. On the left, the text "softmax" is followed by a gray rounded rectangle labeled  $XQK^\top X^\top$  enclosed in large parentheses. This is followed by a teal vertical rectangle labeled  $XV$ , an equals sign, and another gray vertical rectangle. To the right of this rectangle is the text "output  $\in \mathbb{R}^{n \times d}$ ".

# The Transformer Encoder: Key-Query-Value Attention

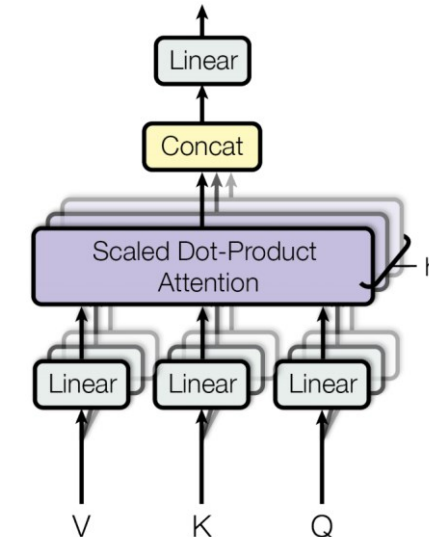
- We saw that self-attention is when keys, queries, and values come from the same source. The Transformer does this in a particular way:
  - Let  $x_1, \dots, x_n$  be input vectors to the Transformer encoder;  $x_i \in \mathbb{R}^d$
- Then keys, queries, values are:
  - $k_i = Kx_i$ , where  $K \in \mathbb{R}^{d \times d}$  is the key matrix.
  - $q_i = Qx_i$ , where  $Q \in \mathbb{R}^{d \times d}$  is the query matrix.
  - $v_i = Vx_i$ , where  $V \in \mathbb{R}^{d \times d}$  is the value matrix.
- These matrices allow *different aspects* of the  $x$  vectors to be used/emphasized in each of the three roles.

# Multi-head attention

- The input word vectors could be the queries, keys and values
  - In other words: the word vectors themselves select each other
  - Word vector stack =  $Q = K = V$
- **Problem:** Only one way for words to interact with one-another
- **Solution:** Multi-head attention
  - First map  $Q, K, V$  into  $h$  many lower dimensional spaces via  $W$  matrices
  - Then apply attention, then concatenate outputs and pipe  $t$

$$\text{MultiHead}(Q, K, V) = \text{Concat}(\text{head}_1, \dots, \text{head}_h) W^O$$

where  $\text{head}_i = \text{Attention}(QW_i^Q, KW_i^K, VW_i^V)$



# Scaled Dot-Product Attention

- **“Scaled Dot Product”** attention aids in training.
- When dimensionality  $d$  becomes large, dot products between vectors tend to become large.
  - Because of this, inputs to the softmax function can be large, making the gradients small.

- Instead of the self-attention function we’ve seen:

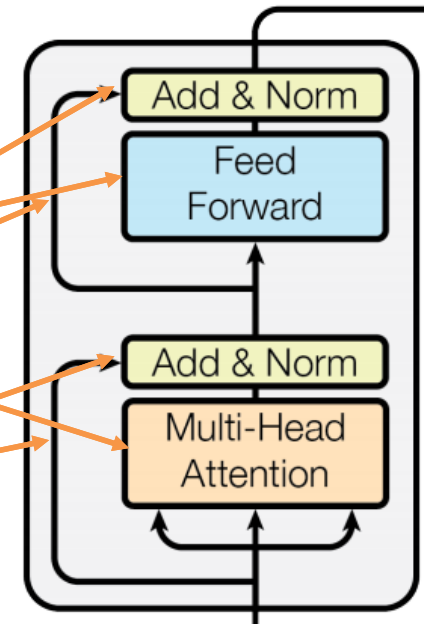
$$\text{output}_\ell = \text{softmax}(XQ_\ell K_\ell^\top X^\top) * XV_\ell$$

- We divide the attention scores by  $\sqrt{d/h}$ , to stop the scores from becoming large just as a function of  $d/h$  (The dimensionality divided by the number of heads.)

$$\text{output}_\ell = \text{softmax}\left(\frac{XQ_\ell K_\ell^\top X^\top}{\sqrt{d/h}}\right) * XV_\ell$$

# Complete transformer block

- Each block has two “sublayers”
  - 1. Multihead attention
  - 2. layer feed-forward Nnet (with relu)
- Each of these two steps also has:
  - Residual (short-circuit) connection and LayerNorm:
  - LayerNorm(x + Sublayer(x))
    - Layernorm changes input to have mean 0 and variance 1, per layer and per training point (and adds two more parameters)

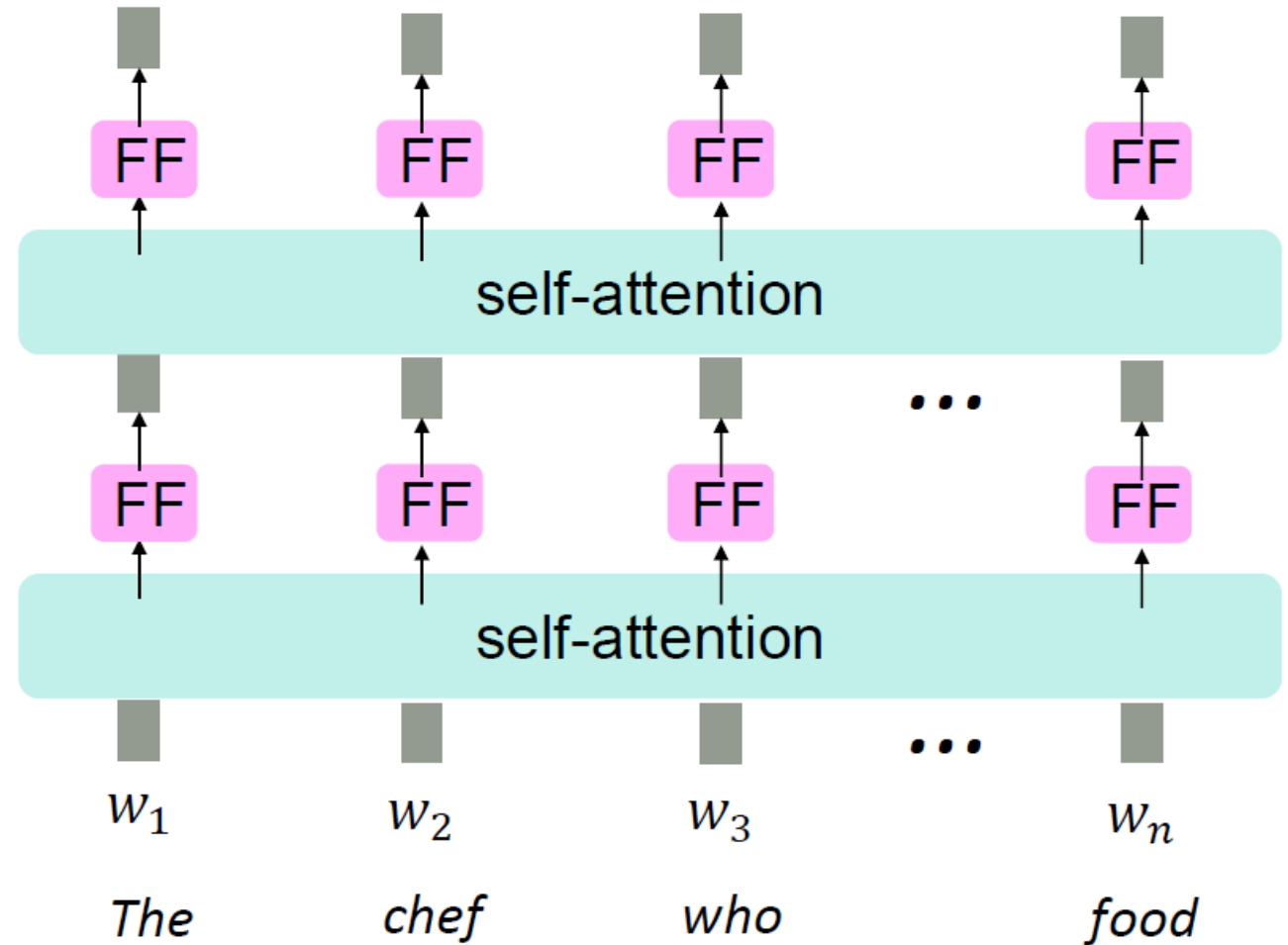


$$\mu^l = \frac{1}{H} \sum_{i=1}^H a_i^l \quad \sigma^l = \sqrt{\frac{1}{H} \sum_{i=1}^H (a_i^l - \mu^l)^2} \quad h_i = f\left(\frac{g_i}{\sigma_i} (a_i - \mu_i) + b_i\right)$$

# Adding nonlinearities in self-attention

- Note that there are no elementwise nonlinearities in self-attention; stacking more self-attention layers just re-averages **value** vectors (Why? Look at the notes!)
- Easy fix: add a **feed-forward network** to post-process each output vector.

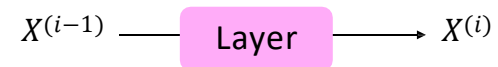
$$\begin{aligned} m_i &= MLP(\text{output}_i) \\ &= W_2 * \text{ReLU}(W_1 \text{output}_i + b_1) + b_2 \end{aligned}$$



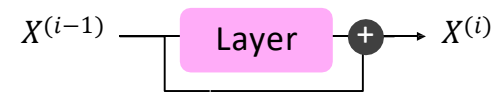
# The Transformer Encoder: Residual connections [[He et al., 2016](#)]

- **Residual connections** are a trick to help models train better.

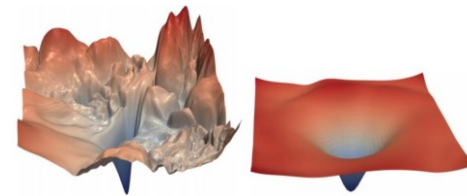
- Instead of  $X^{(i)} = \text{Layer}(X^{(i-1)})$  (where  $i$  represents the layer)



- We let  $X^{(i)} = X^{(i-1)} + \text{Layer}(X^{(i-1)})$  (so we only have to learn “the residual” from the previous layer)



- Residual connections are thought to make the loss landscape considerably smoother (thus easier training!)



[no residuals]

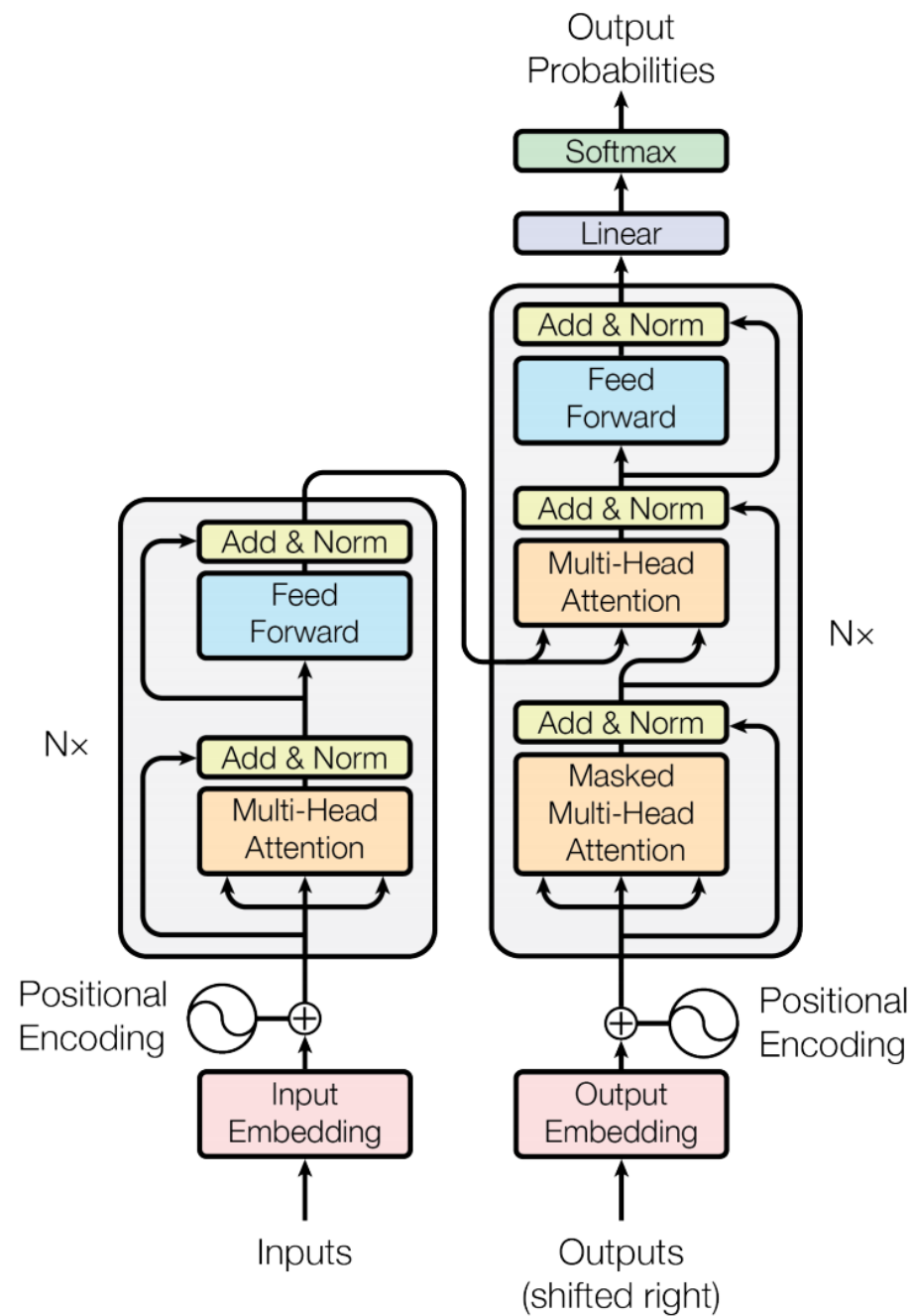
[residuals]

[Loss landscape visualization,  
[Li et al., 2018](#), on a ResNet]

# Layer Normalization [Ba et al., 2016]

- **Problem:** Difficult to train the parameters of a given layer because its input from the layer beneath keeps shifting.
- **Solution:** Reduce uninformative variation by normalizing to **zero mean and standard deviation of one** within each layer.

Blocks are repeated  
N=6 times

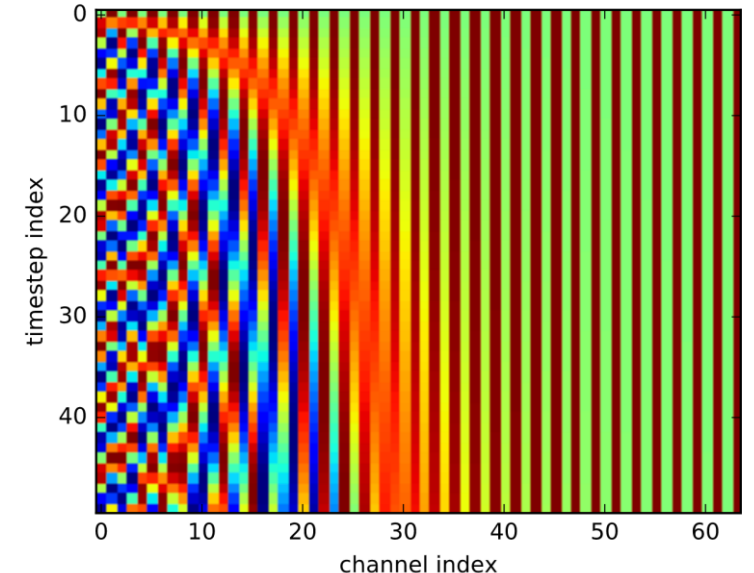


# Encoder Input

- Actual word representations are byte-pair encodings (subword)
  - Rico Sennrich, Barry Haddow, and Alexandra Birch. Neural Machine Translation of Rare Words with Subword Units. ACL 2016.
- Solution: Add “positional encoding” to the input embeddings:  $\mathbf{p}_i$  for  $i \in \mathbb{R}^d$   $i = 1, 2, \dots, n$ 
$$\mathbf{x}_i \leftarrow \mathbf{x}_i + \mathbf{p}_i$$
- Added is a **positional encoding** so same words at different locations have different overall representations (sine and cosine functions of different frequencies ):

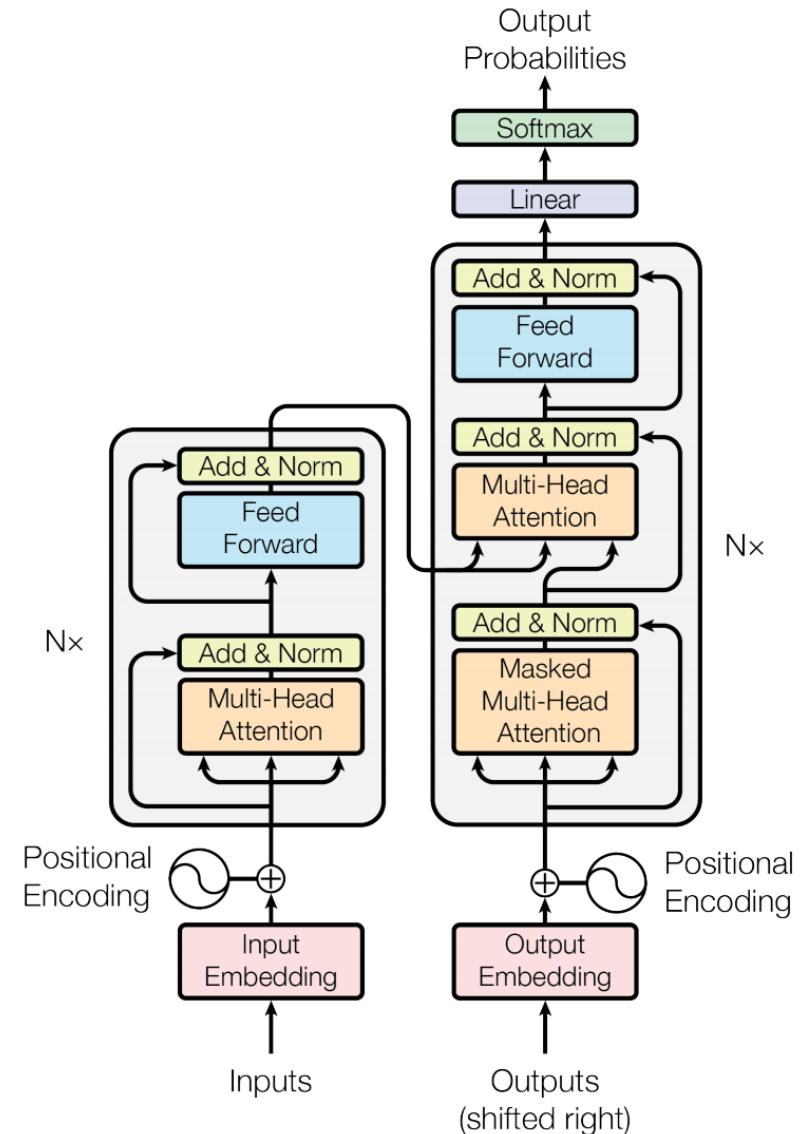
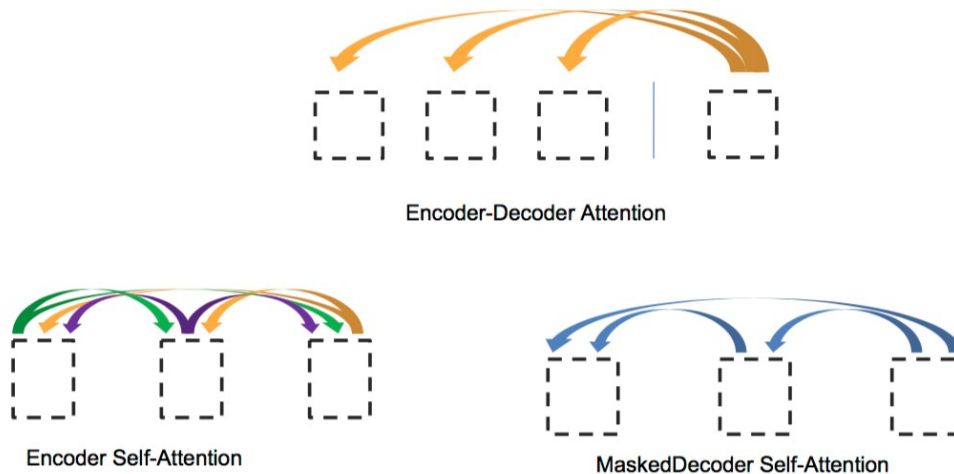
$$PE_{(pos, 2i)} = \sin(pos/10000^{2i/d_{\text{model}}})$$
$$PE_{(pos, 2i+1)} = \cos(pos/10000^{2i/d_{\text{model}}})$$

$pos$  is the position of a word  
 $i$  is the dimension index



# Transformer Decoder

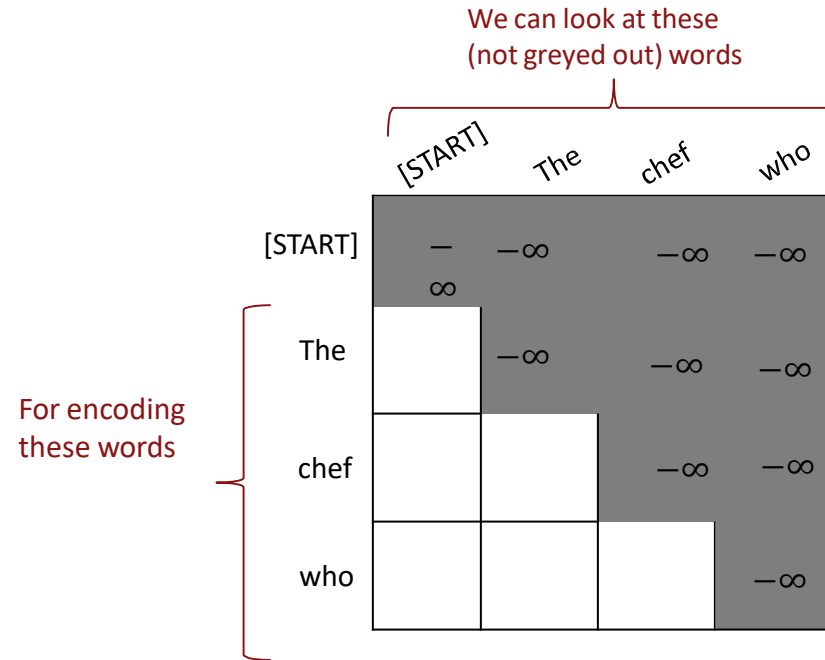
- 2 sublayer changes in decoder
- Masked decoder self-attention
  - Only depends on previous words
- Encoder-Decoder Attention
  - Queries come from previous decoder layer and keys and values come from output of encoder



# Masking the future in self-attention

- To use self-attention in **decoders**, we need to ensure we can't peek at the future.
- At every timestep, we could change the set of **keys and queries** to include only past words. (Inefficient!)
- To enable parallelization, we **mask out attention** to future words by setting attention scores to  $-\infty$ .

$$e_{ij} = \begin{cases} q_i^\top k_j, & j < i \\ -\infty, & j \geq i \end{cases}$$



# Great Results with Transformers

Table 2: The Transformer achieves better BLEU scores than previous state-of-the-art models on the English-to-German and English-to-French newstest2014 tests at a fraction of the training cost.

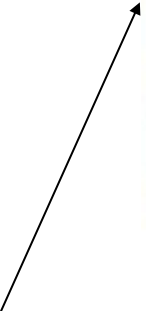
| Model                           | BLEU        |              | Training Cost (FLOPs)                 |                     |
|---------------------------------|-------------|--------------|---------------------------------------|---------------------|
|                                 | EN-DE       | EN-FR        | EN-DE                                 | EN-FR               |
| ByteNet [18]                    | 23.75       |              |                                       |                     |
| Deep-Att + PosUnk [39]          |             | 39.2         |                                       | $1.0 \cdot 10^{20}$ |
| GNMT + RL [38]                  | 24.6        | 39.92        | $2.3 \cdot 10^{19}$                   | $1.4 \cdot 10^{20}$ |
| ConvS2S [9]                     | 25.16       | 40.46        | $9.6 \cdot 10^{18}$                   | $1.5 \cdot 10^{20}$ |
| MoE [32]                        | 26.03       | 40.56        | $2.0 \cdot 10^{19}$                   | $1.2 \cdot 10^{20}$ |
| Deep-Att + PosUnk Ensemble [39] |             | 40.4         |                                       | $8.0 \cdot 10^{20}$ |
| GNMT + RL Ensemble [38]         | 26.30       | 41.16        | $1.8 \cdot 10^{20}$                   | $1.1 \cdot 10^{21}$ |
| ConvS2S Ensemble [9]            | 26.36       | <b>41.29</b> | $7.7 \cdot 10^{19}$                   | $1.2 \cdot 10^{21}$ |
| Transformer (base model)        | 27.3        | 38.1         | <b><math>3.3 \cdot 10^{18}</math></b> |                     |
| Transformer (big)               | <b>28.4</b> | <b>41.8</b>  | $2.3 \cdot 10^{19}$                   |                     |

# Great Results with Transformers

Next, document generation!

| Model                                                         | Test perplexity | ROUGE-L |
|---------------------------------------------------------------|-----------------|---------|
| <i>seq2seq-attention, <math>L = 500</math></i>                | 5.04952         | 12.7    |
| <i>Transformer-ED, <math>L = 500</math></i>                   | 2.46645         | 34.2    |
| <i>Transformer-D, <math>L = 4000</math></i>                   | 2.22216         | 33.6    |
| <i>Transformer-DMCA, no MoE-layer, <math>L = 11000</math></i> | 2.05159         | 36.2    |
| <i>Transformer-DMCA, MoE-128, <math>L = 11000</math></i>      | 1.92871         | 37.9    |
| <i>Transformer-DMCA, MoE-256, <math>L = 7500</math></i>       | 1.90325         | 38.8    |

The old standard



Transformers all the way down.



[[Liu et al., 2018](#)]; WikiSum dataset

# Advantages

- No recurrence: parallel encoding
- Fast training: both encoder and decoder are parallel
- No long range problem:  $O(1)$  for all tokens direct connections
- Three attentions: the model does not have to remember too much
- Multi-head attention allows to pay attention to different aspects

# Open source & Demo

- ViNMT: Neural Machine Translation Toolkit
  - [https://github.com/KCDichDaNgu/KC4.0\\_MultilingualNMT](https://github.com/KCDichDaNgu/KC4.0_MultilingualNMT)
- Neural Machine Translation System:
  - <https://dichmay.itrithuc.vn/>

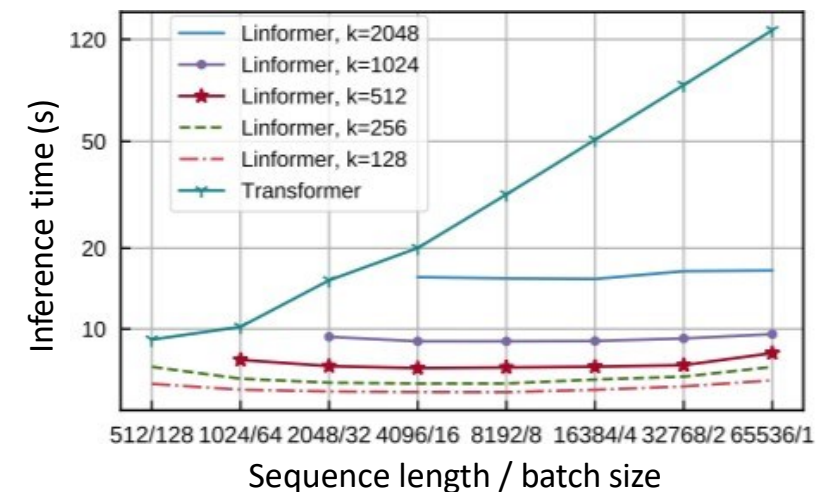
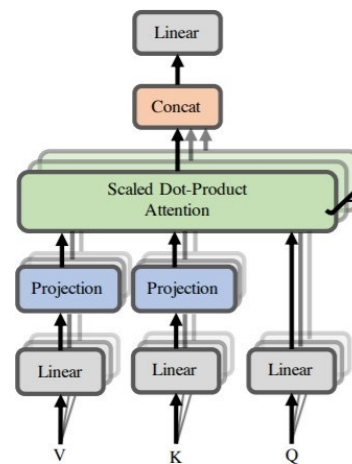
# What would we like to fix about the Transformer?

- **Quadratic compute in self-attention:**
  - Computing all pairs of interactions means our computation grows **quadratically** with the sequence length!
  - For recurrent models, it only grew linearly!
- **Position representations:**
  - Are simple absolute indices the best we can do to represent position?
  - Relative linear position attention [\[Shaw et al., 2018\]](#)
  - Dependency syntax-based position [\[Wang et al., 2019\]](#)

# Recent work on improving on quadratic self-attention cost

- Considerable recent work has gone into the question, *Can we build models like Transformers without paying the  $O(T^2)$  all-pairs self-attention cost?*
- For example, **Linformer** [\[Wang et al., 2020\]](#) ( )

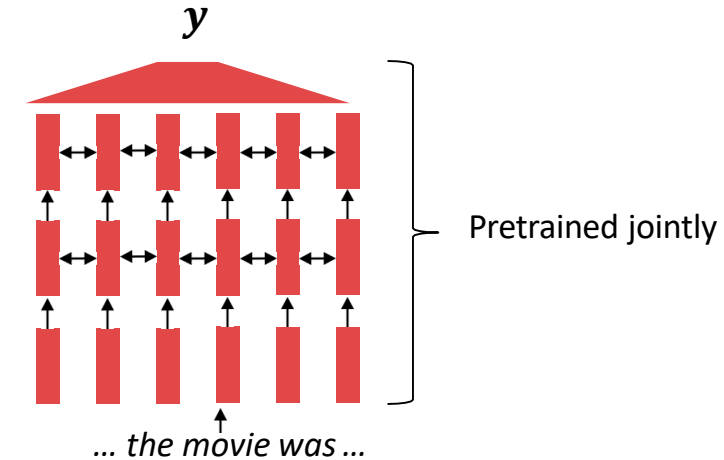
**Key idea:** map the sequence length dimension to a lower-dimensional space for values, keys



# Pretraining models

## In modern NLP:

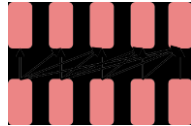
- All (or almost all) parameters in NLP networks are initialized via **pretraining**.
- Pretraining methods hide parts of the input from the model, and train the model to reconstruct those parts.
- This has been exceptionally effective at building strong:
  - **representations of language**
  - **parameter initializations** for strong NLP models.



[This model has learned how to represent entire sentences through pretraining]

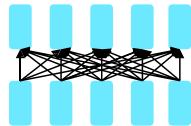
# Pretraining for three types of architectures

The neural architecture influences the type of pretraining, and natural use cases.



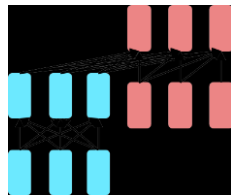
**Decoders**

- Language models! What we've seen so far.
- Nice to generate from; can't condition on future words



**Encoders**

- Gets bidirectional context – can condition on future!
- Wait, how do we pretrain them?



**Encoder-  
Decoders**

- Good parts of decoders and encoders?
- What's the best way to pretrain them?

# Google's BERT (2018)

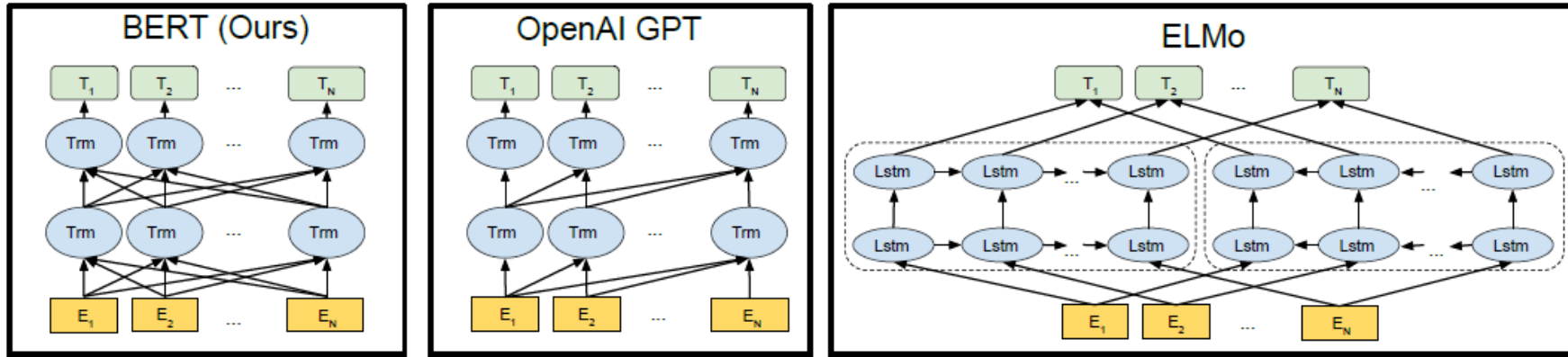


Figure 1: Differences in pre-training model architectures. BERT uses a bidirectional Transformer. OpenAI GPT uses a left-to-right Transformer. ELMo uses the concatenation of independently trained left-to-right and right-to-left LSTM to generate features for downstream tasks. Among three, only BERT representations are jointly conditioned on both left and right context in all layers.

- Training of BERT-BASE was performed on 4 Cloud TPUs in Pod configuration (16 TPU chips total).
- Training of BERT-LARGE was performed on 16 Cloud TPUs (64 TPU chips total).
- Each pre-training took 4 days to complete.

<https://ai.googleblog.com/2018/11/open-sourcing-bert-state-of-art-pre.html>

<https://arxiv.org/pdf/1810.04805.pdf>

# Practice

- Understanding Transformer deeply: <https://jalammar.github.io/illustrated-transformer/>
- Steps for implement Transformer:  
<http://nlp.seas.harvard.edu/annotated-transformer/>
- ViNMT: Neural Machine Translation Toolkit
  - [https://github.com/KCDichDaNgu/KC4.0\\_MultilingualNMT](https://github.com/KCDichDaNgu/KC4.0_MultilingualNMT)

# Conclusion

- Transformers is the state of the art for any application of NLPs.
- Pretraining models for Transformer: Encoder (Bert), Decoder (GPT3, ChatGPT), Encoder-Decoder (T5, Bart)
- Now, Transformer is success for **Foundation Models**.